Week 6 Assignment

Repository Link:
https://github.com/Hughes-Tyler/AnalyticsCapstone

Report Link:
https://github.com/Hughes-Tyler/AnalyticsCapstone/blob/main/notebooks/main.ipynb

This week we trained our first model on the processed startup dataset. We chose a Random Forest classifier, evaluated three hyperparameter variations that move from shallow to deep, calculated training and validation metrics for each, and selected the variation that generalizes best.

We chose Random Forest because it fits the shape of our data without requiring a ton of extra work. The final dataset has 5 continuous PCA components and 46 binary country and category flags, and Random Forest splits on both kinds of columns directly, so we didn't need to scale anything. Startup outcomes also don't move in a straight line with any single feature. How much a company raised only matters in combination with how quickly it raised it, and decision trees pick up that kind of interaction on their own. Averaging many trees trained on random subsets also keeps any one tree's noise from dominating the forest, which matters more than usual here since several of our features (the funding pace feature in particular) are weak or sparse. Finally, Random Forest supports `class_weight='balanced'`, which we decided on in Week 5 to handle our class imbalance instead of SMOTE, so mistakes on rare outcomes like IPOs cost more during training without us having to invent synthetic rows.

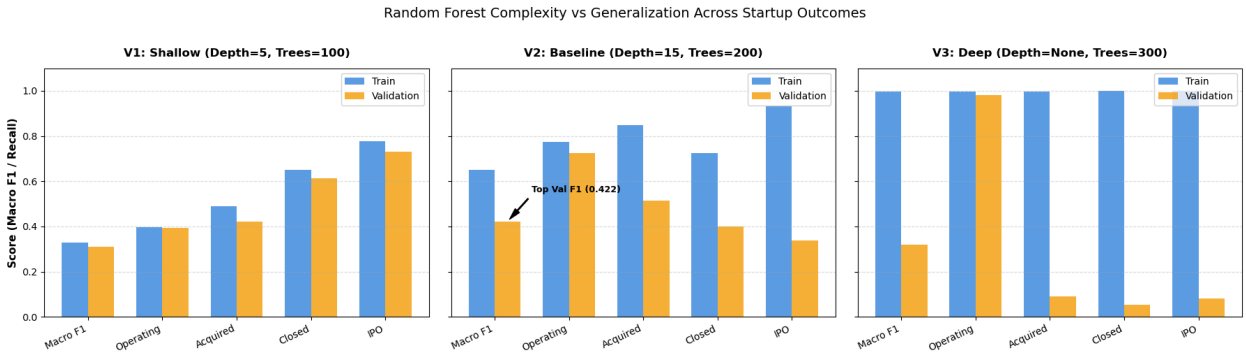
Random Forest's complexity is controlled almost entirely by two knobs, and we varied both together to move from a simple model to a more complex one. `max_depth` limits how many splits a tree is allowed to make before it has to stop, so a shallow tree can only separate a handful of broad groups while an unrestricted tree can keep splitting until it isolates individual rows. `n_estimators` is the number of trees being averaged; more trees don't add complexity to any single tree, but they make the forest's predictions more stable as depth increases, since each tree gets to vote. We chose these two specifically because they are the standard way to trade off underfitting against overfitting in a tree ensemble, and this week's goal was to see that tradeoff play out directly. We tested:

- V1 - Shallow: `n_estimators=100, max_depth=5`. Heavily restricts how complex each tree can get, which should underfit but generalize well.
- V2 - Baseline: `n_estimators=200, max_depth=15` A middle ground setting we expect to balance fit and generalization.
- V3 - Deep: `n_estimators=300, max_depth=None` Trees grow until every leaf is pure, which should fit the training data closely and is the variation most at risk of overfitting.

Every variation used `class_weight='balanced'` and the same `random_state`, so complexity is the only thing that changes between them. We trained each one on our 29,782 row training set and generated predictions on both that set and the 9,928 row validation set, saving the trained models and a combined predictions file so metrics could be calculated without retraining anything.

Our target is heavily imbalanced (81.7% operating, 8.3% acquired, 8.0% closed, 2.0% IPO), so raw accuracy is a pretty bad grading measure. AS mentioned in previous weeks/discussion posts, a model that predicts "operating" for every company would score 81.7% while finding zero acquisitions, closures, or IPOs. We instead graded each variation on macro F1 and class recall. Macro F1 averages the F1 score across all four classes equally, so the model can't look good by only getting the majority class right, and it drops immediately if the model misses the minority classes. Per class recall tells us something macro F1 can't by itself, which is the exact share of companies in each outcome the model actually catches, which is what lets us check whether the balanced weights are doing anything for acquisitions, closures, and IPOs specifically.

Week 6 metrics (macro F1 and recall per class):



As the forest gets deeper, training performance rises greatly while validation performance doesn't follow along. Training macro F1 climbs from 0.329 (V1) to 0.649 (V2) to 0.996 (V3), but validation macro F1 goes 0.311, 0.422, 0.319, peaking at V2 and falling back at V3. The train minus validation gap in macro F1 grows from 0.019 (V1) to 0.228 (V2) to 0.677 (V3), so each step toward more complexity buys training performance that doesn't transfer.

V1 just plainly underfits. Its training and validation scores are nearly identical (0.329 vs. 0.311), so it isn't overfitting, it's just too simple to learn the patterns in the first place. With only five levels per tree and balanced class weights pushing it toward the minority classes, it ends up over predicting acquired, closed, and IPO at the expense of operating, finding only 39% of operating companies. Its validation accuracy, 42.1%, is way below the 81.7% we'd get by guessing operating every time, which is the clearest sign that it's sacrificing the majority class without getting enough back from the minority ones.

V3 has the exact the opposite problem. It fits the training set almost perfectly (macro F1 0.996, every class recall at or above 0.997), but validation macro F1 collapses to 0.319. Recall on acquired, closed, and IPO falls by 0.91 to 0.95 points between training and validation, and on validation the model labels almost every company as operating (recall 0.981). Its 81.5% validation accuracy is about the same as the all operating baseline mentioned before, meaning it has essentially memorized the training rows and learned almost nothing generalizable about the other outcomes.

V2 sits between the two. Its training scores rise nicely over V1 (macro F1 0.649, IPO recall 0.931) but validation drops to 0.422, so there is overfitting here too, especially in the minority classes: IPO recall falls from 0.931 to 0.338, closed from 0.725 to 0.399, and acquired

from 0.849 to 0.515, while operating recall barely moves (0.773 to 0.724). Despite that gap, V2 has the best validation macro F1 of the three, about 0.10 higher than both V1 and V3, and it's the only variation that keeps reasonable operating recall (0.724) while still finding roughly half of acquired companies (0.515). V1 and V3 land at a similar macro F1 for opposite reasons: V1 spreads its predictions across the minority classes at the cost of operating, while V3 collapses almost entirely to operating. V1 does have the highest validation recall for closed (0.613) and IPO (0.732) though, but that comes from over predicting those classes rather than from actually separating them well.

In the end, we selected V2 (Baseline: `n_estimators=200, max_depth=15`) as the winning model. It has the highest validation macro F1 (0.422, vs. 0.311 for V1 and 0.319 for V3), which is the metric we chose specifically for this imbalanced, multi class problem. V3's training score is the highest by a wide margin (0.996), but that doesn't hold up on validation, where it misses 91-95% of every outcome except operating. V1 is stable across training and validation but too weak to be useful, missing most of the majority class in exchange for only a modest improvement on the minority classes. V2 is not a strong model in absolute terms, since it still misses about two thirds of IPOs on validation and still shows a moderate train validation gap, but it is the best of the three on validation data, and it's the natural starting point for further tuning.